Structured Query Language

Complete Reference Notes

Prepared for
Sameeksha

✓ DDL & DML Commands	✓ Joins & Subqueries	✓ Aggregate Functions
✓ Constraints	✓ Views & Indexes	✓ Transactions & ACID
✓ Stored Procedures	✓ Window Functions	✓ Query Optimisation

1. Introduction to SQL

SQL (Structured Query Language) is the standard language for managing and manipulating relational databases. It was developed by IBM in the 1970s, standardised by ANSI/ISO, and is today supported by every major RDBMS including MySQL, PostgreSQL, Oracle, SQL Server, and SQLite.

SQL is declarative: you describe *what* you want, not *how* to get it. The database engine figures out the most efficient execution plan.

1.1 SQL Sub-languages

Sub-language	Full Form	Key Commands
DDL	Data Definition Language	CREATE, ALTER, DROP, TRUNCATE, R NAME
DML	Data Manipulation Language	SELECT, INSERT, UPDATE, DELETE
DCL	Data Control Language	GRANT, REVOKE
TCL	Transaction Control Language	COMMIT, ROLLBACK, SAVEPOINT
DQL	Data Query Language	SELECT (sometimes separated)

1.2 RDBMS Concepts

- **Table (Relation):** A two-dimensional structure of rows (tuples) and columns (attributes).
- **Primary Key:** A column or combination of columns that uniquely identifies every row.
- **Foreign Key:** A column that references the primary key of another table, enforcing referential integrity.
- **Schema:** The logical container that organises tables, views, and other objects.
- **NULL:** Represents an unknown or missing value — not zero, not empty string.

2. Data Definition Language (DDL)

2.1 CREATE TABLE

Creates a new table in the database. You define column names, data types, and constraints.

```
CREATE TABLE students (
student_id INT PRIMARY KEY AUTO_INCREMENT,
first_name VARCHAR(50) NOT NULL,
last_name VARCHAR(50) NOT NULL,
email VARCHAR(100) UNIQUE,
dob DATE,
gpa DECIMAL(3,2) DEFAULT 0.00,
dept_id INT,
FOREIGN KEY (dept_id) REFERENCES departments(dept_id)
);
```

2.2 ALTER TABLE

Modifies an existing table structure — add/drop columns, modify data types, add constraints.

```
-- Add a column
ALTER TABLE students ADD COLUMN phone VARCHAR(15);

-- Modify column data type
ALTER TABLE students MODIFY COLUMN phone VARCHAR(20);

-- Rename a column (MySQL 8+)
ALTER TABLE students RENAME COLUMN phone TO contact_number;

-- Drop a column
ALTER TABLE students DROP COLUMN contact_number;

-- Add a constraint
ALTER TABLE students ADD CONSTRAINT chk_gpa CHECK (gpa BETWEEN 0 AND 10);
```

2.3 DROP & TRUNCATE

```
-- Permanently removes table and all its data
DROP TABLE students;

-- Removes all rows but keeps table structure; resets auto-increment
TRUNCATE TABLE students;

-- Drop table only if it exists (prevents error)
DROP TABLE IF EXISTS students;
```

■ *TRUNCATE is DDL (cannot be rolled back in most DBs); DELETE is DML (can be rolled back).*

2.4 Common Data Types

Category	Type	Description
Numeric	INT / BIGINT	Whole numbers (4 / 8 bytes)
Numeric	DECIMAL(p,s)	Exact fixed-point number
Numeric	FLOAT / DOUBLE	Approximate floating-point
String	CHAR(n)	Fixed-length string
String	VARCHAR(n)	Variable-length string (max n)
String	TEXT	Large text (up to 65 535 chars)
Date/Time	DATE	YYYY-MM-DD
Date/Time	DATETIME / TIMESTAMP	YYYY-MM-DD HH:MM:SS
Other	BOOLEAN	TRUE / FALSE (stored as 1/0)
Other	BLOB	Binary large object

3. Constraints

Constraints enforce rules on data to maintain accuracy and integrity.

PRIMARY KEY

Uniquely identifies each record. A table can have only one primary key.

```
student_id INT PRIMARY KEY
```

NOT NULL

Ensures a column cannot store NULL values.

```
first_name VARCHAR(50) NOT NULL
```

UNIQUE

Ensures all values in a column (or combination) are distinct.

```
email VARCHAR(100) UNIQUE
```

FOREIGN KEY

Links two tables and enforces referential integrity.

```
FOREIGN KEY (dept_id) REFERENCES departments(dept_id)
ON DELETE CASCADE ON UPDATE CASCADE
```

CHECK

Restricts the values that can be placed in a column.

```
age INT CHECK (age >= 18)
```

DEFAULT

Provides a default value when none is specified during INSERT.

```
status VARCHAR(10) DEFAULT 'active'
```

4. Data Manipulation Language (DML)

4.1 SELECT — The Foundation

```
-- Basic syntax
SELECT column1, column2
FROM table_name
WHERE condition
GROUP BY column
HAVING group_condition
ORDER BY column [ASC|DESC]
LIMIT n OFFSET m;

-- Select all columns
SELECT * FROM students;

-- Distinct values
SELECT DISTINCT dept_id FROM students;

-- Column alias
SELECT first_name AS "First Name", gpa * 10 AS score FROM students;
```

4.2 WHERE Clause & Operators

```
SELECT * FROM students WHERE gpa > 8.0;

-- Logical operators
SELECT * FROM students WHERE dept_id = 2 AND gpa >= 7;
SELECT * FROM students WHERE dept_id = 1 OR dept_id = 3;
SELECT * FROM students WHERE NOT gpa < 5;

-- BETWEEN (inclusive)
SELECT * FROM students WHERE gpa BETWEEN 7.0 AND 9.0;

-- IN
SELECT * FROM students WHERE dept_id IN (1, 2, 5);

-- LIKE (pattern matching)
SELECT * FROM students WHERE first_name LIKE 'S%'; -- starts with S
SELECT * FROM students WHERE email LIKE '%@gmail.com';
SELECT * FROM students WHERE last_name LIKE '_arma'; -- _ = any single char

-- IS NULL / IS NOT NULL
SELECT * FROM students WHERE email IS NULL;
```

4.3 INSERT

```
-- Insert single row
INSERT INTO students (first_name, last_name, email, gpa, dept_id)
VALUES ('Sameeksha', 'Sharma', 'sameeksha@email.com', 9.2, 1);

-- Insert multiple rows
INSERT INTO students (first_name, last_name, gpa)
VALUES ('Alice', 'Roy', 8.5),
('Bob', 'Das', 7.0),
('Carol', 'Nair', 9.0);

-- Insert from SELECT
INSERT INTO top_students (student_id, gpa)
SELECT student_id, gpa FROM students WHERE gpa > 9.0;
```

4.4 UPDATE

```
-- Update specific rows
UPDATE students
SET gpa = 9.5, email = 'new@email.com'
WHERE student_id = 101;

-- Update with subquery
UPDATE students
SET dept_id = (SELECT dept_id FROM departments WHERE dept_name = 'CS')
WHERE student_id = 102;
```

■ *Always use WHERE with UPDATE/DELETE. Without it you modify/delete ALL rows.*

4.5 DELETE

```
-- Delete specific rows
DELETE FROM students WHERE gpa < 4.0;

-- Delete all rows (DML — can be rolled back)
DELETE FROM students;
```

5. Aggregate Functions & GROUP BY

Aggregate functions compute a single result from a set of rows. They are used with SELECT and often with GROUP BY.

Function	Description	Example
COUNT(*)	Total number of rows	COUNT(*) returns 500
COUNT(col)	Non-NULL values in column	COUNT(email)
SUM(col)	Sum of numeric column	SUM(salary)
AVG(col)	Average of numeric column	AVG(gpa)
MIN(col)	Smallest value	MIN(dob)
MAX(col)	Largest value	MAX(gpa)
GROUP_CONCAT	Concatenate values (MySQL)	GROUP_CONCAT(name)

```
-- Count students per department
SELECT dept_id, COUNT(*) AS total_students, AVG(gpa) AS avg_gpa
FROM students
GROUP BY dept_id
HAVING AVG(gpa) > 7.5 -- HAVING filters GROUPS (not rows)
ORDER BY avg_gpa DESC;

-- WHERE vs HAVING
-- WHERE filters ROWS before grouping
-- HAVING filters GROUPS after grouping
SELECT dept_id, COUNT(*) AS cnt
FROM students
WHERE gpa > 6 -- filter individual rows first
GROUP BY dept_id
HAVING cnt > 10; -- then filter groups
```

6. Joins

A JOIN combines rows from two or more tables based on a related column.

INNER JOIN

Returns rows with matching values in BOTH tables.

LEFT JOIN

All rows from left table; NULLs for unmatched right rows.

RIGHT JOIN

All rows from right table; NULLs for unmatched left rows.

FULL OUTER JOIN

All rows from both tables; NULLs where no match.

CROSS JOIN

Cartesian product — every row from left × every row from right.

SELF JOIN

A table joined with itself (uses aliases).

```
-- Sample schema
-- students(student_id, name, dept_id)
-- departments(dept_id, dept_name)
-- courses(course_id, course_name, dept_id)
-- enrollments(student_id, course_id, grade)

-- INNER JOIN
SELECT s.first_name, d.dept_name
FROM students s
INNER JOIN departments d ON s.dept_id = d.dept_id;

-- LEFT JOIN (all students, even those without a dept)
SELECT s.first_name, d.dept_name
FROM students s
LEFT JOIN departments d ON s.dept_id = d.dept_id;

-- Multiple JOINs
SELECT s.first_name, c.course_name, e.grade
FROM students s
JOIN enrollments e ON s.student_id = e.student_id
JOIN courses c ON e.course_id = c.course_id
WHERE e.grade = 'A';

-- SELF JOIN (find students in same dept)
SELECT a.first_name AS student1, b.first_name AS student2, a.dept_id
```

```
FROM students a
JOIN students b ON a.dept_id = b.dept_id AND a.student_id < b.student_id;

-- CROSS JOIN
SELECT s.first_name, c.course_name
FROM students s CROSS JOIN courses c;
```

7. Subqueries

A subquery is a query nested inside another query. It can appear in SELECT, FROM, WHERE, or HAVING clauses.

7.1 Subquery in WHERE

```
-- Students with GPA above average
SELECT first_name, gpa
FROM students
WHERE gpa > (SELECT AVG(gpa) FROM students);

-- Students enrolled in 'Data Structures'
SELECT first_name FROM students
WHERE student_id IN (
SELECT student_id FROM enrollments
WHERE course_id = (SELECT course_id FROM courses
WHERE course_name = 'Data Structures')
);
```

7.2 Correlated Subquery

Executes once per row of the outer query. References outer query columns.

```
-- Students with GPA higher than their dept average
SELECT s1.first_name, s1.gpa, s1.dept_id
FROM students s1
WHERE s1.gpa > (SELECT AVG(s2.gpa)
FROM students s2
WHERE s2.dept_id = s1.dept_id);
```

7.3 EXISTS / NOT EXISTS

```
-- Students who have enrolled in at least one course
SELECT first_name FROM students s
WHERE EXISTS (
SELECT 1 FROM enrollments e WHERE e.student_id = s.student_id
);

-- Students who haven't enrolled in any course
SELECT first_name FROM students s
WHERE NOT EXISTS (
SELECT 1 FROM enrollments e WHERE e.student_id = s.student_id
);
```

7.4 Subquery in FROM (Derived Table)

```
SELECT dept_id, avg_gpa
FROM (SELECT dept_id, AVG(gpa) AS avg_gpa
```

```
FROM students
GROUP BY dept_id) AS dept_avg
WHERE avg_gpa > 8.0;
```

8. Set Operations

Combine results of two or more SELECT statements. Both queries must have the same number of columns with compatible data types.

Operator	Description	Duplicates?
UNION	Combine results, remove duplicates	No
UNION ALL	Combine results, keep duplicates	Yes
INTERSECT	Rows present in BOTH results	No
EXCEPT / MINUS	Rows in first but not second	No

```
-- UNION (unique students from two batches)
SELECT student_id FROM batch_2023
UNION
SELECT student_id FROM batch_2024;

-- UNION ALL (preserve all, including dupes)
SELECT course_name FROM cs_courses
UNION ALL
SELECT course_name FROM it_courses;

-- INTERSECT (students enrolled in BOTH courses)
SELECT student_id FROM course_101_enrollments
INTERSECT
SELECT student_id FROM course_102_enrollments;

-- EXCEPT (students in 101 but NOT in 102)
SELECT student_id FROM course_101_enrollments
EXCEPT
SELECT student_id FROM course_102_enrollments;
```

9. Built-in Functions

9.1 String Functions

```
SELECT UPPER('sameeksha'); -- SAMEEKSHA
SELECT LOWER('HELLO'); -- hello
SELECT LENGTH('SQL'); -- 3
SELECT CHAR_LENGTH('SQL'); -- 3 (for multibyte safety)
SELECT CONCAT('Hello', ' ', 'World');-- Hello World
SELECT SUBSTRING('Database', 1, 4); -- Data
SELECT TRIM(' hello '); -- hello
SELECT LTRIM(' hi'); -- hi
SELECT RTRIM('hi '); -- hi
SELECT REPLACE('2024-01-15','-','/');-- 2024/01/15
SELECT LOCATE('SQL', 'Learn SQL'); -- 7
SELECT LPAD('7', 3, '0'); -- 007
SELECT RPAD('Hi', 5, '!'); -- Hi!!!
SELECT REVERSE('SQL'); -- LQS
```

9.2 Numeric Functions

```
SELECT ABS(-15); -- 15
SELECT CEIL(4.3); -- 5
SELECT FLOOR(4.9); -- 4
SELECT ROUND(4.567, 2); -- 4.57
SELECT MOD(17, 5); -- 2
SELECT POWER(2, 10); -- 1024
SELECT SQRT(144); -- 12
SELECT TRUNCATE(9.999, 1); -- 9.9
```

9.3 Date & Time Functions

```
SELECT NOW(); -- 2024-06-01 14:35:22
SELECT CURDATE(); -- 2024-06-01
SELECT CURTIME(); -- 14:35:22
SELECT YEAR(NOW()); -- 2024
SELECT MONTH('2024-06-01'); -- 6
SELECT DAY('2024-06-01'); -- 1
SELECT DAYNAME('2024-06-01'); -- Saturday
SELECT DATE_ADD(NOW(), INTERVAL 7 DAY);
SELECT DATE_SUB(NOW(), INTERVAL 1 MONTH);
SELECT DATEDIFF('2024-12-31','2024-01-01'); -- 365
SELECT DATE_FORMAT(NOW(), '%d/%m/%Y'); -- 01/06/2024
```

9.4 Conditional Functions

```
-- IF
SELECT first_name, IF(gpa >= 8, 'Distinction', 'Pass') AS result
```

```
FROM students;

-- CASE expression
SELECT first_name,
CASE
WHEN gpa >= 9 THEN 'Outstanding'
WHEN gpa >= 7.5 THEN 'Excellent'
WHEN gpa >= 6 THEN 'Good'
ELSE 'Average'
END AS grade_band
FROM students;

-- COALESCE (returns first non-NULL)
SELECT COALESCE(email, phone, 'No contact') AS contact FROM students;

-- NULLIF (returns NULL if both args equal)
SELECT NULLIF(dept_id, 0) FROM students;

-- IFNULL (MySQL)
SELECT IFNULL(gpa, 0.0) FROM students;
```

10. Views

A view is a named, stored SQL query. It acts like a virtual table — you can SELECT from it, and sometimes INSERT/UPDATE/DELETE through it.

Benefits of Views

- Simplifies complex queries by hiding joins and subqueries.
- Provides a security layer — expose only specific columns/rows.
- Ensures consistent results across the application.
- Does NOT store data (except materialised views in some DBs).

```
-- Create a view
CREATE VIEW student_dept_view AS
SELECT s.student_id, s.first_name, s.last_name, s.gpa, d.dept_name
FROM students s
JOIN departments d ON s.dept_id = d.dept_id;

-- Query the view like a table
SELECT * FROM student_dept_view WHERE dept_name = 'Computer Science';

-- Create or replace
CREATE OR REPLACE VIEW high_achievers AS
SELECT * FROM students WHERE gpa >= 9.0;

-- Drop a view
DROP VIEW IF EXISTS student_dept_view;
```

11. Indexes

An index is a data structure (typically a B-tree or hash) that speeds up data retrieval. The trade-off: faster reads but slower writes and extra storage.

Index Type	Description
PRIMARY	Automatically created for primary key; clustered in InnoDB.
UNIQUE	Ensures uniqueness + speeds lookups.
INDEX / KEY	Non-unique index for faster searches.
FULLTEXT	For full-text search on large text columns.
COMPOSITE	Index on multiple columns — order matters.

```
-- Create index
CREATE INDEX idx_gpa ON students(gpa);

-- Unique index
CREATE UNIQUE INDEX idx_email ON students(email);

-- Composite index (covers queries filtering on both columns)
CREATE INDEX idx_dept_gpa ON students(dept_id, gpa);

-- View indexes on a table
SHOW INDEX FROM students;

-- Drop index
DROP INDEX idx_gpa ON students;
```

■ *Use EXPLAIN SELECT ... to check whether a query uses an index.*

12. Transactions & ACID Properties

A transaction is a sequence of SQL operations treated as a single unit of work — either all succeed or none do.

12.1 ACID Properties

Property	Meaning
Atomicity	All operations succeed or all are rolled back.
Consistency	Data moves from one valid state to another.
Isolation	Concurrent transactions do not interfere with each other.
Durability	Committed changes persist even after a crash.

12.2 Transaction Commands

```
-- Start a transaction (implicitly started on first DML in many DBs)
START TRANSACTION; -- or BEGIN;

-- DML operations
UPDATE accounts SET balance = balance - 5000 WHERE acc_id = 1;
UPDATE accounts SET balance = balance + 5000 WHERE acc_id = 2;

-- If everything is OK
COMMIT;

-- If something went wrong
ROLLBACK;

-- Savepoints
SAVEPOINT sp1;
UPDATE students SET gpa = 9.5 WHERE student_id = 10;
ROLLBACK TO SAVEPOINT sp1; -- undo only since sp1
RELEASE SAVEPOINT sp1; -- remove savepoint

-- Auto-commit control
SET autocommit = 0; -- disable auto-commit
SET autocommit = 1; -- enable auto-commit
```

12.3 Isolation Levels

Isolation Level	Dirty Read	Non-Repeatable Read	Phantom Read
READ UNCOMMITTED	Yes	Yes	Yes
READ COMMITTED	No	Yes	Yes

Isolation Level	Dirty Read	Non-Repeatable Read	Phantom Read
REPEATABLE READ	No	No	Yes
SERIALIZABLE	No	No	No

```
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
```

13. Stored Procedures & Functions

13.1 Stored Procedure

A stored procedure is a reusable block of SQL code saved in the database. It can accept IN/OUT parameters and execute complex logic.

```
DELIMITER //

CREATE PROCEDURE get_dept_students(IN p_dept_id INT)
BEGIN
SELECT student_id, first_name, gpa
FROM students
WHERE dept_id = p_dept_id
ORDER BY gpa DESC;
END //

DELIMITER ;

-- Call it
CALL get_dept_students(2);

-- Procedure with OUT parameter
DELIMITER //
CREATE PROCEDURE get_avg_gpa(IN p_dept INT, OUT p_avg DECIMAL(4,2))
BEGIN
SELECT AVG(gpa) INTO p_avg FROM students WHERE dept_id = p_dept;
END //
DELIMITER ;

CALL get_avg_gpa(1, @avg);
SELECT @avg;
```

13.2 User-Defined Function

```
DELIMITER //
CREATE FUNCTION grade_label(p_gpa DECIMAL(3,2))
RETURNS VARCHAR(15)
DETERMINISTIC
BEGIN
IF p_gpa >= 9.0 THEN RETURN 'Outstanding';
ELSEIF p_gpa >= 7.5 THEN RETURN 'Excellent';
ELSEIF p_gpa >= 6.0 THEN RETURN 'Good';
ELSE RETURN 'Average';
END IF;
END //
DELIMITER ;
```

```
-- Use in query
SELECT first_name, gpa, grade_label(gpa) AS grade FROM students;
```

14. Triggers

A trigger is SQL code that automatically fires BEFORE or AFTER an INSERT, UPDATE, or DELETE operation on a table.

```
-- Log every salary update
CREATE TABLE salary_log (
log_id INT AUTO_INCREMENT PRIMARY KEY,
emp_id INT,
old_salary DECIMAL(10,2),
new_salary DECIMAL(10,2),
changed_at DATETIME DEFAULT NOW()
);

DELIMITER //
CREATE TRIGGER after_salary_update
AFTER UPDATE ON employees
FOR EACH ROW
BEGIN
IF OLD.salary <> NEW.salary THEN
INSERT INTO salary_log(emp_id, old_salary, new_salary)
VALUES (OLD.emp_id, OLD.salary, NEW.salary);
END IF;
END //
DELIMITER ;

-- BEFORE INSERT trigger (auto-set default)
DELIMITER //
CREATE TRIGGER before_student_insert
BEFORE INSERT ON students
FOR EACH ROW
BEGIN
IF NEW.gpa IS NULL THEN
SET NEW.gpa = 0.00;
END IF;
END //
DELIMITER ;

-- Drop trigger
DROP TRIGGER IF EXISTS after_salary_update;
```

15. Window Functions

Window functions perform calculations across a 'window' of rows related to the current row — without collapsing rows like GROUP BY does.

```
-- Syntax
function_name(column) OVER (
[PARTITION BY col]
[ORDER BY col]
[ROWS/RANGE BETWEEN ...]
)
```

15.1 Ranking Functions

```
SELECT first_name, gpa, dept_id,
RANK() OVER (PARTITION BY dept_id ORDER BY gpa DESC) AS rnk,
DENSE_RANK() OVER (PARTITION BY dept_id ORDER BY gpa DESC) AS dense_rnk,
ROW_NUMBER() OVER (PARTITION BY dept_id ORDER BY gpa DESC) AS row_num,
NTILE(4) OVER (ORDER BY gpa DESC) AS quartile
FROM students;
```

15.2 Aggregate Window Functions

```
SELECT first_name, gpa, dept_id,
SUM(gpa) OVER (PARTITION BY dept_id) AS dept_gpa_sum,
AVG(gpa) OVER (PARTITION BY dept_id) AS dept_avg,
COUNT(*) OVER (PARTITION BY dept_id) AS dept_size,
SUM(gpa) OVER (ORDER BY student_id ROWS UNBOUNDED PRECEDING) AS running_sum
FROM students;
```

15.3 Lead & Lag

```
SELECT student_id, gpa,
LAG(gpa, 1) OVER (ORDER BY student_id) AS prev_gpa,
LEAD(gpa, 1) OVER (ORDER BY student_id) AS next_gpa,
gpa - LAG(gpa, 1) OVER (ORDER BY student_id) AS gpa_diff
FROM students;
```

16. Common Table Expressions (CTE)

A CTE (WITH clause) creates a temporary named result set that can be referenced within a single SQL statement. It improves readability and allows recursion.

16.1 Non-Recursive CTE

```
WITH dept_avg AS (
SELECT dept_id, AVG(gpa) AS avg_gpa
FROM students
GROUP BY dept_id
),
high_dept AS (
SELECT dept_id FROM dept_avg WHERE avg_gpa > 8.0
)
SELECT s.first_name, s.gpa, d.dept_name
FROM students s
JOIN departments d ON s.dept_id = d.dept_id
WHERE s.dept_id IN (SELECT dept_id FROM high_dept);
```

16.2 Recursive CTE

Useful for hierarchical data like org charts, file systems, or category trees.

```
-- Print numbers 1 to 10
WITH RECURSIVE counter(n) AS (
SELECT 1 -- anchor member
UNION ALL
SELECT n + 1 FROM counter WHERE n < 10 -- recursive member
)
SELECT n FROM counter;

-- Org chart (employees and their managers)
WITH RECURSIVE org_chart(emp_id, name, manager_id, lvl) AS (
SELECT emp_id, name, manager_id, 1
FROM employees WHERE manager_id IS NULL -- top-level
UNION ALL
SELECT e.emp_id, e.name, e.manager_id, oc.lvl + 1
FROM employees e
JOIN org_chart oc ON e.manager_id = oc.emp_id
)
SELECT CONCAT(REPEAT(' ', lvl - 1), name) AS hierarchy
FROM org_chart ORDER BY lvl, emp_id;
```

17. Query Optimisation

17.1 EXPLAIN / EXPLAIN ANALYZE

```
EXPLAIN SELECT * FROM students WHERE gpa > 8.0;
-- Shows: id, select_type, table, type, key, rows, Extra

-- Important 'type' values (best to worst):
-- const → eq_ref → ref → range → index → ALL (full table scan)

EXPLAIN ANALYZE SELECT ...; -- Actually runs & shows real timings (MySQL 8+)
```

17.2 Optimisation Tips

- **Use indexes on WHERE, JOIN, and ORDER BY columns.**
- **Avoid SELECT *** — fetch only needed columns.
- **Avoid functions on indexed columns** in WHERE: *WHERE YEAR(created_at) = 2024* prevents index use. Use range instead: *WHERE created_at BETWEEN '2024-01-01' AND '2024-12-31'*.
- **Use LIMIT** to reduce rows fetched.
- **Prefer JOINs over correlated subqueries** for large datasets.
- **Avoid leading wildcards** in LIKE: *LIKE '%SQL'* cannot use a B-tree index.
- **Use composite indexes** wisely — left-most prefix rule.
- **Analyse slow queries** with the slow query log and EXPLAIN.

18. Normalization

Normalization is the process of organising data to reduce redundancy and improve integrity by dividing tables and defining relationships.

1NF — First Normal Form

Each column holds atomic (indivisible) values. No repeating groups. Each row is unique (has a primary key).

2NF — Second Normal Form

Must be in 1NF. Every non-key column is fully dependent on the ENTIRE primary key (eliminates partial dependency).

3NF — Third Normal Form

Must be in 2NF. No non-key column depends on another non-key column (eliminates transitive dependency).

BCNF — Boyce-Codd NF

Stronger version of 3NF. For every functional dependency $X \rightarrow Y$, X must be a superkey.

4NF / 5NF

Deal with multi-valued and join dependencies — important in specialised data modelling.

19. DCL & TCL Commands

DCL — Data Control Language

```
-- Grant privileges
GRANT SELECT, INSERT ON company_db.students TO 'sameeksha'@'localhost';
GRANT ALL PRIVILEGES ON company_db.* TO 'admin'@'%';

-- Revoke privileges
REVOKE INSERT ON company_db.students FROM 'sameeksha'@'localhost';

-- Create user
CREATE USER 'sameeksha'@'localhost' IDENTIFIED BY 'SecurePass@123';

-- Show grants
SHOW GRANTS FOR 'sameeksha'@'localhost';
```

TCL — Transaction Control Language

```
START TRANSACTION;
INSERT INTO orders VALUES (1001, 'Laptop', 75000);
SAVEPOINT after_insert;

UPDATE inventory SET stock = stock - 1 WHERE product = 'Laptop';

-- Oops, wrong product
ROLLBACK TO SAVEPOINT after_insert;

UPDATE inventory SET stock = stock - 1 WHERE product = 'Laptop Pro';
COMMIT;
```

20. SQL Quick Reference Cheat-Sheet

Command / Clause	Purpose	Key Notes
SELECT	Retrieve data	Use aliases, DISTINCT, *
FROM	Source table(s)	Can be subquery (derived table)
WHERE	Filter rows	Runs BEFORE GROUP BY
GROUP BY	Aggregate rows	Columns not in agg must be listed
HAVING	Filter groups	Runs AFTER GROUP BY
ORDER BY	Sort results	ASC (default) or DESC
LIMIT/OFFSET	Paginate results	MySQL; FETCH NEXT in SQL Server
JOIN	Combine tables	INNER, LEFT, RIGHT, FULL, CROSS
UNION	Merge SELECT results	Removes dupes; UNION ALL keeps
INSERT	Add rows	Specify columns explicitly
UPDATE	Modify rows	Always use WHERE!
DELETE	Remove rows	Always use WHERE!
CREATE TABLE	Define new table	Specify types + constraints
ALTER TABLE	Modify table structure	ADD/DROP/MODIFY columns
DROP TABLE	Delete table + data	Irreversible!
TRUNCATE	Empty table fast	DDL; resets AUTO_INCREMENT
CREATE VIEW	Store named query	Virtual table
CREATE INDEX	Speed up queries	Slower writes; check with EXPLAIN
TRANSACTION	Group DML atomically	COMMIT / ROLLBACK
GRANT/REVOKE	Manage permissions	Principle of least privilege

Happy Learning, Sameeksha!

Master SQL step by step — practice every query, read EXPLAIN plans, and build real projects.